Studio Nx

Backend(FastAPI/Python) 39 — delete_profile_image()

2026-10-12

Backend(FastAPI/Python) 39 — `delete_profile_image()`

항목	내용
파일	`studionx/backend/app/services/user_service.py`
작업	P1-08 회원 정보: 프로필 사진 지우기 (PRD 01 FR-106)
작성	Claude (AI), 2026-10-07, 사용자 검토
이전 · 다음	38 `save_profile_image()` · 40 `profile_image_file()`

1. 하는 일

내 프로필 사진을 지웁니다. DB의 `profile_image_path`를 비우고, 저장된 파일도 지웁니다. 그러면 화면은 닉네임 첫 글자가 들어간 동그라미로 돌아갑니다.

상황	결과
사진이 있음	경로는 `None`이 되고, 파일이 지워지며, 응답의 사진 주소는 `null`입니다
사진이 없음, 두 번 누름	오류 없이 `None` 그대로입니다
DB에는 경로가 있는데 파일은 이미 없음	오류 없이 경로를 `None`으로 바꿉니다
DB 저장 실패	오류를 그대로 전달하고, **파일은 남겨 둡니다**

2. 언제 쓰나

쓰는 곳	
`DELETE /users/me/profile-image` (42)	userInfo.html의 Remove 버튼(P1-10)을 누르면 이 함수를 부르고, 36번으로 응답합니다
P1-09 회원 탈퇴 `withdraw()`	템플릿 6번 "사진 파일 지우기 (delete_profile_image 재사용, commit은 마지막에 한 번)"에 해당합니다. 탈퇴 상태를 바꾼 **뒤 맨 마지막에** 이 함수를 부르면, 이 안의 `commit`이 탈퇴 상태와 사진 삭제를 **함께** 저장합니다

3. 코드

import (파일 맨 위: 38번과 같고, 새로 추가한 것은 없습니다). 이 함수는 **`get_settings`**를 사용합니다.

함수

```
def delete_profile_image(db: Session, user: User) -> User:
    old_relative = user.profile_image_path
    user.profile_image_path = None
    db.commit()
    if old_relative:
        (get_settings().storage_dir / old_relative).unlink(missing_ok=True)
    db.refresh(user)
    return user
```

문법 풀이

식	뜻
`old_relative = user.profile_image_path`	지우기 전에 경로를 따로 기억해 둡니다. 바로 아래에서 `None`으로 바꾸기 때문입니다
`get_settings().storage_dir / old_relative`	실제 파일의 위치입니다 (`studionx/storage/profiles/….jpg`)
`unlink(missing_ok=True)`	파일을 지우고, 이미 없어도 오류를 내지 않습니다
`db.refresh(user)`	DB가 바꾼 `updated_at`까지 다시 읽습니다

4. 흐름

1. 지금 경로를 기억해 두고 DB의 경로를 비웁니다.
2. 저장(commit)합니다. 실패하면 여기서 오류가 나고, 파일은 그대로 남습니다.
3. 저장에 성공했고 예전 경로가 있었다면 파일을 지웁니다.
4. 다시 읽어서 돌려줍니다.

5. 왜 이렇게

선택	이유
DB를 먼저, 파일은 나중에 처리 (템플릿은 "파일 삭제 → DB")	38번과 같은 이유입니다. 템플릿 순서대로라면 DB 저장이 실패했을 때, DB는 사진이 있다고 하는데 파일은 없어서 사진이 깨집니다. 이 순서라면 실패해도 사진이 그대로 보입니다 (반대로 파일 삭제가 실패하면 파일만 남을 뿐, 화면에는 영향이 없습니다)
오류 없이 끝나는 경우를 넓게 둠	Remove를 빠르게 두 번 누르거나, 파일을 손으로 지운 경우에도 사용자에게 오류를 보여 줄 이유가 없습니다
`commit`을 함수 안에서 함 (매개변수로 끄지 않음)	템플릿은 탈퇴에서 "commit은 마지막에 한 번"이라고 했습니다. 탈퇴가 이 함수를 **맨 마지막에** 부르면 같은 결과가 됩니다. 켜고 끄는 매개변수를 두면 "파일을 언제 지우는가"가 부르는 쪽의 책임이 되어 더 복잡해집니다
38번의 `rollback` 같은 정리 코드가 없음	지워야 할 새 파일이 없기 때문입니다. 오류가 나면 요청이 끝날 때 세션이 닫히면서 되돌려집니다

6. 시험 (`tests/test_user_service.py`에 5개를 추가해서 모두 44개, 실제 MySQL과 임시 폴더로 실행)

시험	확인한 것
사진 지우기	같은 객체를 돌려주고, 경로, 응답 주소, DB가 모두 `None`이며, 파일이 지워져 폴더가 비었습니다
사진 없음, 두 번 누름	오류가 없습니다
파일이 이미 없음	오류 없이 경로가 `None`이 됩니다
다른 회원의 사진	지워지지 않습니다
DB 저장 실패	오류가 그대로 전달되고, 파일과 내용은 그대로입니다

전체 시험 278개가 통과했고, `ruff` 검사도 통과했습니다.